

# Greedy Algorithm

## Analysis and Design of Algorithms



## The Core Idea

01

**Make a choice: At each step, pick the best available option based on a specific criterion.**

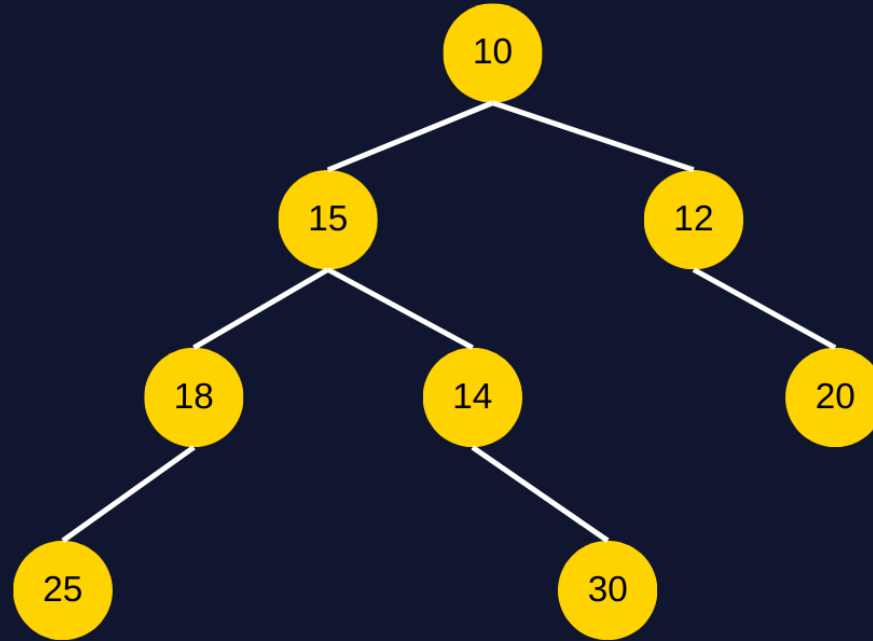
02

**Proceed to the next step: Move forward and repeat the process until the problem is solved.**

03

**Check the outcome: The algorithm arrives at the solution that is either optimal or close to optimal**

# The Core Idea



Greedy Algorithm Example

# Applying the greedy algorithm

- Start at the root (10).
  - Left child: 15
  - Right child: 12
  - Greedy choice: Pick 15 (since it's the largest).



# Applying the greedy algorithm

- Start at the root (10).
  - Left child: 15
  - Right child: 12
  - Greedy choice: Pick 15 (since it's the largest).
- Move to node (15).
  - Left child: 18
  - Right child: 14
  - Greedy choice: Pick 18 (since it's the largest).



# Applying the greedy algorithm

- Start at the root (10).
  - Left child: 15
  - Right child: 12
  - Greedy choice: Pick 15 (since it's the largest).
- Move to node (15).
  - Left child: 18
  - Right child: 14
  - Greedy choice: Pick 18 (since it's the largest).
- Move to node (18).
  - Left child: 25
  - No right child.
  - Greedy choice: Pick 25 (since it's the only option).



# Applying the greedy algorithm

- Final Path Chosen by Greedy Algorithm:
  - $10 \rightarrow 15 \rightarrow 18 \rightarrow 25$
  - Total sum:  $10 + 15 + 18 + 25 = 68$



# Does the greedy approach find the optimal solution?

- Let's check all possible paths to confirm that the greedy algorithm was correct:
  - Path:  $10 \rightarrow 15 \rightarrow 18 \rightarrow 25 \rightarrow$  Sum = 68
  - Path:  $10 \rightarrow 15 \rightarrow 14 \rightarrow 30 \rightarrow$  Sum = 69 (Not possible since 30 is unreachable)
  - Path:  $10 \rightarrow 12 \rightarrow 20 \rightarrow$  Sum = 42





Let's change the structure a bit to show how making a local greedy choice can lead to a less optimal overall solution.



# Applying the greedy algorithm

- Start at the root (10)
  - Left child: 15
  - Right child: 12
  - Greedy choice: Pick 15 (since it's larger)



# Applying the greedy algorithm

- Start at the root (10)
  - Left child: 15
  - Right child: 12
  - Greedy choice: Pick 15 (since it's larger)
- Move to node (15)
  - Left child: 18
  - Right child: 14
  - Greedy choice: Pick 18 (since it's larger)



# Applying the greedy algorithm

- Start at the root (10)
  - Left child: 15
  - Right child: 12
  - Greedy choice: Pick 15 (since it's larger)
- Move to node (15)
  - Left child: 18
  - Right child: 14
  - Greedy choice: Pick 18 (since it's larger)
- Move to node (18)
  - Left child: 25
  - No right child
  - Greedy choice: Pick 25 (only option)



# Applying the greedy algorithm

- The final path chosen:
  - $10 \rightarrow 15 \rightarrow 18 \rightarrow 25$
  - Total sum = 68



# Does the greedy approach find the optimal solution?

- let's explore all possible paths:
  - Greedy path:
    - $10 \rightarrow 15 \rightarrow 18 \rightarrow 25$
    - Sum = 68
- Alternative path:
  - $10 \rightarrow 15 \rightarrow 14 \rightarrow 30$
  - Sum =  $10 + 15 + 14 + 30 = 69$  Higher than greedy choice!
- Other path:
  - $10 \rightarrow 12 \rightarrow 20$
  - Sum = 42 (Lower)



# Example 1: Classroom Scheduling

- **Greedy Strategy**

- Sort classes by **finish time** (earliest finish first).
- Select a class if it doesn't overlap with the previously selected one.



## Example 1: Classroom Scheduling

- $[(1, 3), (3, 5), (5, 7), (8, 9)]$





## Example 1: Classroom Scheduling

| Step | Class<br>(start,end) | Last End | Condition  | Action        | Selected                  |
|------|----------------------|----------|------------|---------------|---------------------------|
| 1    | (1,3)                | 0        | $1 \geq 0$ | <b>Select</b> | [(1,3)]                   |
| 2    | (2,4)                | 3        | $2 \geq 3$ | Skip          | [(1,3)]                   |
| 3    | (3,5)                | 3        | $3 \geq 3$ | <b>Select</b> | [(1,3),(3,5)]             |
| 4    | (0,6)                | 5        | $0 \geq 5$ | Skip          | [(1,3),(3,5)]             |
| 5    | (5,7)                | 5        | $5 \geq 5$ | <b>Select</b> | [(1,3),(3,5),(5,7)]       |
| 6    | (8,9)                | 7        | $8 \geq 7$ | <b>Select</b> | [(1,3),(3,5),(5,7),(8,9)] |

## Example 2: Job Sequencing with Deadlines

- Each job has:
  - a profit
  - a deadline
  - takes exactly 1 time slot
- Goal: maximize total profit while completing jobs before deadlines.



## Example 2: Job Sequencing with Deadlines

- **Greedy Strategy**

- Sort jobs by profit (descending)
- Assign each job to the latest possible free slot before its deadline.



## Example 2: Job Sequencing with Deadlines

| Job | Profit | Deadline | Try Slot | Free? | Action              | Schedule                |
|-----|--------|----------|----------|-------|---------------------|-------------------------|
| A   | 100    | 2        | slot 2   | Yes   | Place A             | [None, None, 'A', None] |
| C   | 27     | 2        | slot 2   | No    | Try slot 1 →<br>Yes | Place C                 |
| D   | 25     | 1        | slot 1   | No    | Skip                | [None, 'C', 'A', None]  |
| B   | 19     | 1        | slot 1   | No    | Skip                | [None, 'C', 'A', None]  |
| E   | 15     | 3        | slot 3   | Yes   | Place E             | [None, 'C', 'A', 'E']   |

# Example 3: Fractional Knapsack Problem

- **Problem**
  - You want to fill a knapsack with maximum value.  
Each item has:
    - value
    - weight
- You are allowed to **take fractions** of items.



# Example 3: Fractional Knapsack Problem

- **Greedy Strategy**

- Compute value/weight ratio (density)
- Sort items by highest ratio
- Take as much as possible until the bag is full



## Step 1: Compute value/weight ratio

| Item | Value | Weight | Ratio |
|------|-------|--------|-------|
| 1    | 60    | 10     | 6     |
| 2    | 100   | 20     | 5     |
| 3    | 120   | 30     | 4     |



## Step 2: Tracing the algorithm

| Step | Item (v,w) | Ratio | Remaining Cap | Action                            | Value Added | Total Value |
|------|------------|-------|---------------|-----------------------------------|-------------|-------------|
| 1    | (60,10)    | 6     | 50            | Take full                         | +60         | 60          |
| 2    | (100,20)   | 5     | 40            | Take full                         | +100        | 160         |
| 3    | (120,30)   | 4     | 20            | Take <b>20/30</b><br><b>= 2/3</b> | +4×20 = 80  | 240         |

